# CSS

Colan Biemer

# A Web Page

```
<h1>Colan's Coffee Blog</h1>
<p>Here is some text about the purpose of my made-up blog.</p>
<h2>Visited</h2>
<ul>
  <li>Recreo - Dynamite</li>
  <li>Nero - at least it looks nice.</li>
  <li>Starbucks - it's fine.</li>
</ul>
```

- HTML
- HTML + CSS

# What Is CSS?

**CSS** stands for **Cascading Style Sheets**.

- Style Sheet: a list of rules for how the page should look
- Cascading: when more than one rule could apply to an element, later or more specific rules override earlier ones

## Example: A Red Paragraph

```
<p style="color: red;">This paragraph is red.</p>
```

This paragraph is red.

The `style` attribute puts CSS directly on one element. This is **inline** CSS.

## Exercise: Blue and Red

Write two paragraphs. Make the first one blue and the second one red.

Paragraph 1

Paragraph 2

# Exercise: Blue and Red (Continued)

```
<p style="color: blue;">Paragraph 1</p>
<p style="color: red;">Paragraph 2</p>
```

## Exercise: Red, Green, Blue

Add a third paragraph. Make them red, green, and blue, in that order.

Paragraph 1

Paragraph 2

Paragraph 3

# Exercise: Red, Green, Blue (Continued)

```
<p style="color: red;">Paragraph 1</p>
<p style="color: green;">Paragraph 2</p>
<p style="color: blue;">Paragraph 3</p>
```

## Brief Aside: Named Colors

CSS currently defines 148 named colors: red, blue, green, rebeccapurple, antiquewhite, burlywood, darkgoldenrod, ... MDN lists them all.

We aren't limited to just those, though.

## Any Color with `rgb()`

```
<p style="color: rgb(201, 123, 58);">Paragraph 1</p>
```

Paragraph 1

`rgb(red, green, blue)`: how much of each, from `0` (none) to `255` (full).

You will also see hex ( `#c97b3a` ) and `hsl(...)`. Same idea, but we won't be covering these in any depth.

# Problem: What Color Is Each Paragraph?

```
<p style="color: rgb(255, 0, 0);">Paragraph 1</p>
<p style="color: rgb(0, 255, 0);">Paragraph 2</p>
<p style="color: rgb(0, 0, 255);">Paragraph 3</p>
<p style="color: rgb(0, 0, 0);">Paragraph 4</p>
<p style="color: rgb(128, 128, 128);">Paragraph 5</p>
<p style="color: rgb(255, 255, 255);">Paragraph 6</p>
```

## Solution: What Color Is Each Paragraph?

Paragraph 1: rgb(255, 0, 0)

Paragraph 2: rgb(0, 255, 0)

Paragraph 3: rgb(0, 0, 255)

Paragraph 4: rgb(0, 0, 0)

Paragraph 5: rgb(128, 128, 128)

## Exercise: All `red` with `rgb()`

Make all three paragraphs red with `rgb()`.

Paragraph 1

Paragraph 2

Paragraph 3

## Exercise: All `red` with `rgb()` (Continued)

```
<p style="color: rgb(255, 0, 0);">Paragraph 1</p>
<p style="color: rgb(255, 0, 0);">Paragraph 2</p>
<p style="color: rgb(255, 0, 0);">Paragraph 3</p>
```

## Problem: All `red`

What if we had 40 paragraphs, and we decided we wanted them to be red? We would have to do each paragraph individually:

```
<p style="color: rgb(255, 0, 0);">Paragraph 1</p>
<p style="color: rgb(255, 0, 0);">Paragraph 2</p>
<p style="color: rgb(255, 0, 0);">Paragraph 3</p>
...
<p style="color: rgb(255, 0, 0);">Paragraph 40</p>
```

If we ever want to change the color, we have a ton of work ahead of us. (Find and replace all is a great tool but can get messy.)

## Problem: All `red` (Continued)

Inline CSS works for one-off tweaks (e.g. this one link should be a different color). To style a whole group of elements at once, we need a way to select a group.

## Solution: `<style>`

A `<style>` element in the `<head>` holds rules for the whole page. This is **internal** CSS.

```
<style>
  [SELECTOR] {
    [property]: [VALUE];
  }
</style>
```

# Selector in CSS

```
[SELECTOR] {
  [property]: [VALUE];
}
```

The **selector** picks which elements the rule applies to. Any tag name works: `p`, `div`, `h1`, `li`, `a`, ...

## Original Problem: All Red

What internal-CSS will make all paragraphs red?

## Solution: All Red

```
p {
  color: red;
}
```

Paragraph 1

Paragraph 2

Paragraph 3

# New Problem: Inline Versus Internal CSS

```
<head>
  <style>
    p {
      color: red;
    }
  </style>
</head>
<body>
  <p style="color: blue;">Paragraph 1</p>
  <p>Paragraph 2</p>
</body>
```

What will the color of paragraph 1 be? What about paragraph 2?

## Inline > Internal

Paragraph 1

Paragraph 2

Inline-CSS will overwrite internal CSS. The only exception is when you use `!important`.

## Example: `!important`

```
<head>
  <style>
    p {
      color: red !important;
    }
  </style>
</head>
<body>
  <p style="color: blue;">Paragraph 1</p>
  <p>Paragraph 2</p>
</body>
```

Now, what will the output look like?

## Result: `!important`

Paragraph 1

Paragraph 2

`!important` overrides inline-CSS.

## Exercise: Purple Divs

How do I make text in every `<div>` purple?

## Exercise: Purple Divs (Continued)

```
div {
  color: purple;
}
```

## Exercise: Everything Blue

How can I make all the text in the whole document blue?

## Exercise: Everything Blue (Continued)

```
body {
  color: blue;
}
```

All the text is inside the `<body>` element. Every element inside `<body>` **inherits** the font color.

## Exercise: All Red, Except Divs

How can I make all the font red, except `<div>` font, which will be purple?

# Exercise: All Red, Except Divs (Continued)

```
body {
  color: red;
}

div {
  color: purple;
}
```

`<body>` says red, so any font inside it will be red, *unless* a tag inside `<body>` (the `<div>` in this case) has a `color` rule of its own.

# Software Nomenclature: Parents and Children

```
<body>
  <ul>
    <li>Recreo</li>
    <li>Nero</li>
  </ul>
</body>
```

- An element that sits inside another element is that element's **child**
- The element it sits inside is its **parent**
- Each `<li>` is a child of `<ul>`. `<ul>` is the parent of both, and is itself a child of `<body>` which is a child of `<html>`

## Inheritance

- When a rule sets a property on an element, its children start out with the same value
- Style `<ul>` and every `<li>` inside it matches. Style `<body>` and everything on the page matches
- CSS calls this **inheritance**: values flow down from parent to child

## Inheritance Is a Fallback

- A child only uses the inherited value when no rule matches it directly
- If a rule matches the child itself, that rule wins, and the inherited value is ignored

## What Inherits?

Generally, text- and color-related properties inherit (`color`, `font-size`, `line-height`) and box properties do not (`border`, `margin`, `width`). When in doubt, look it up: every MDN property page has an **Inherited: yes/no** row.

## Problem: Red Body, Blue Link

```
body {
  color: red;
}
```

```
<body>
  <p>Read the <a href="reviews.html">reviews</a>.</p>
</body>
```

`<a>` is a child of `<p>`, which is a child of `<body>`. What color is the word "reviews"?

## Solution: Red Body, Blue Link

Read the reviews.

Blue. The paragraph inherits red from `<body>`, but the link does not: a rule matches `<a>` directly, so inheritance is skipped.

We never wrote a rule for `<a>`. The browser did.

## The Browser's Default Stylesheet

Every browser ships with its own stylesheet (the **user agent stylesheet**). It gave the HTML-only page on the first slide its big heading and bullets. A simplified piece of it:

```
h1 {
  font-size: 2em;
}
a {
  color: blue;
  text-decoration: underline;
}
ul {
  list-style: disc;
  padding-left: 40px;
}
```

## Your CSS Sits on Top

- The browser's stylesheet applies first; yours is layered after it
- If one of your rules matches an element, it beats the browser's rule for that element
- To change a default, write a rule that matches the element (e.g. `a { color: red; }`)

## Example: Changing Font Size and Family for Italicized Text

- How do we make every `<i>` one and a half times bigger? (`font-size`)
- How do we make every `<i>` use a serif font like Georgia? (`font-family`)

## Brief Aside: Fonts

`font-family` takes a list. The browser uses the first font it has installed from the list. End the list with a generic family, which every browser has:

- `serif`: letters with little feet (Georgia, Times New Roman)
- `sans-serif`: no feet (Arial, Helvetica)
- `monospace`: every letter the same width (Courier New)

## Example: Changing Font Size and Family for Italicized Text (Continued)

```
i {
  font-size: 150%;
  font-family: Georgia, serif;
}
```

Example *italicized* text.

## Example: Link Background Color

- How do we give every link a yellow background? (`background-color`)

## Example: Link Background Color (Continued)

```
a {
  background-color: yellow;
}
```

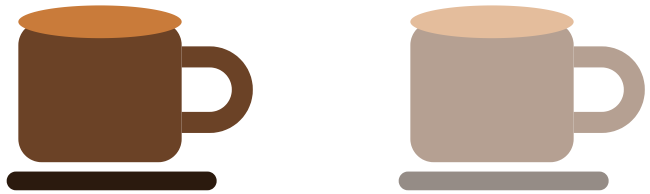
Documentation on MDN is good.

## Example: Low-Opacity Images

- How do we make every image half transparent? (`opacity`, from `0` to `1`)

# Example: Low-Opacity Images (Continued)

```
img {
  opacity: 0.5;
}
```



- Left: `opacity: 1` (the default).
- Right: `opacity: 0.5`.
- `0` is invisible, `1` is solid, and anything between lets the background show through.

## There Are Hundreds of Properties

You have seen `color`, `font-size`, `font-family`, `background-color`, and `opacity`, but there are hundreds more. Few memorize them, but you don't have to. Instead, focus on learning how to use CSS (`selector { property: value; }`) and then look things up as you go. The MDN CSS reference is an excellent resource for this.

## Problem: Two Kinds of Paragraphs

How do we give every paragraph a `padding` of `100px`, but a few of them a `padding` of `50px`?

- `padding` is the space between an element's text and the edge of its box. More on it later.
- `px` is short for pixels.

## Solution: Two Kinds of Paragraphs

```
<head>
  <style>
    p {
      padding: 100px;
    }
  </style>
</head>
<body>
  <p>Paragraph 1</p>
  <p style="padding: 50px;">Paragraph 2</p>
  <p>Paragraph 3</p>
</body>
```

This works. Inline beats internal, so paragraph 2 gets `50px`.

## Problem: 40 Paragraphs, Again

```
<p>Paragraph 1</p>
<p style="padding: 50px;">Paragraph 2</p>
<p>Paragraph 3</p>
<p style="padding: 50px;">Paragraph 4</p>
...
<p style="padding: 50px;">Paragraph 40</p>
```

Say that ten of the paragraphs have the `50px` styling, but we decide `50px` should really be `40px`. Then we have to make ten updates, and if we change our mind again, we have to make ten more. There needs to be a better way.

## Solution: Classes

```
p {
  padding: 100px;
}
.compact {
  padding: 50px;
}
```

```
<p>Paragraph 1</p>
<p class="compact">Paragraph 2</p>
<p>Paragraph 3</p>
```

Name the special paragraphs with the `class` attribute; `.compact` (with the dot) selects them. Change `50px` to `40px` once and all ten follow.

# Solution: Classes (Continued)

Paragraph 1

Paragraph 2

## Selecting by Class

```
.[CLASS_NAME] {
  ...
}
```

## Problem: `<span>` with Class to Be Bold and/or Italicized

I want to make classes for `<span>` (`"bold"` and `"italicized"`) that let me make inline text bold and/or italicized. What is the CSS for this?

| Property | Values | Result |
| --- | --- | --- |
| `font-weight` | `normal`, `bold`, `100` to `900` | **how heavy the letters are** |
| `font-style` | `normal`, `italic` | *whether the letters slant* |

## Solution: `<span>` with Class to Be Bold and/or Italicized (But Not Right)

```
.bold {
  font-weight: bold;
}

.italicized {
  font-style: italic;
}
```

Why is this not right? "I want to make classes for `<span>` (`"bold"` and `"italicized"`) that let me make inline text bold and/or italicized." I want a class that only activates in a `<span>`.

## Solution: `<span>` with Class to Be Bold and/or Italicized

```
/* This is a CSS comment */
span.bold {
  font-weight: bold;
}

span.italicized {
  font-style: italic;
}
```

- No space in `span.bold`: a `<span>` that has class `bold`.
- With a space, `span .bold` means an element with class `bold` *inside* a `<span>`.

## Solution: `<span>` with Class to Be Bold and/or Italicized (Continued)

```
<p>
  Recreo is <span class="bold">dynamite</span>, Nero is
  <span class="italicized">fine</span>, Starbucks is
  <span class="bold italicized">everywhere</span>.
</p>
```

Recreo is **dynamite**, Nero is *fine*, Starbucks is ***everywhere***.

Two classes (`<span class="bold italicized">`) on one element get both rules.

# Problem: No Space

```
p.bold {
  font-weight: bold;
}
```

```
<p class="bold">One</p>
<p><span class="bold">Two</span></p>
```

Which word is bold?

## Solution: No Space

**One**

Two

No space means **and**: `p.bold` is a `<p>` that also has class `bold`. "Two" has the class, but on a `<span>`, so it stays plain.

# Problem: Space

```
p .underline {
  text-decoration: underline;
}
```

```
<p class="underline">One</p>
<p><span class="underline">Two</span></p>
<p class="underline"><span>Three</span></p>
```

Which words are underlined?

## Solution: Space

One

Two

Three

A space means **inside**: `p .underline` is anything with class `underline` inside a `<p>`. "One" and "Three" put the class on the `<p>` itself, and nothing *inside* them has it, so both stay plain.

## Problem: Spans and Links

- How do we give every `<span>`, every `<a>`, and everything with class `blackAndYellow` a black background with yellow text? (`background-color`, `color`)

# Solution: Spans and Links

```
span {
  background-color: black;
  color: yellow;
}

a {
  background-color: black;
  color: yellow;
}

.blackAndYellow {
  background-color: black;
  color: yellow;
}
```

This works, but if we ever make a change, we have to change the other two.

# Solution: Spans and Links (Continued)

```
span,
a,
.blackAndYellow {
  background-color: black;
  color: yellow;
}
```

```
<p>This is a <span>span</span> and this is a <a href="#">link</a>.</p>
<p class="blackAndYellow">This paragraph has class blackAndYellow.</p>
```

This is a span and this is a link.

This paragraph has class blackAndYellow.

## Commas Explained

```
span,
a,
.blackAndYellow {
  background-color: black;
  color: yellow;
}
```

A comma makes a list: the rule applies to every `<span>`, every `<a>`, and everything with class `blackAndYellow`. So far: no space means **and**, a space means **inside**, a comma means **each of these**.

## Problem: Blue Divs and Paragraphs

- How do we give every `<div>` and every `<p>` a blue background? (`background-color`)

# Solution: Blue Divs and Paragraphs

```
div,
p {
  background-color: blue;
}
```

```
<div>This is a div.</div>
<p>This is a paragraph.</p>
<span>This is a span.</span>
```

This is a div.

This is a paragraph.

This is a span.

## Review: Jump to a Section

Imagine we have a blog and we want to allow users to jump to the header:

```
<a [WHAT GOES HERE?]>Visited</a>

<!--Code in Between -->

<h2 [WHAT GOES HERE?]>Visited</h2>
```

# Review: Jump to a Section (Solution)

```
<a href="#visited">Visited</a>

<!--Code in Between -->

<h2 id="visited">Visited</h2>
```

`<a>` (anchor) lets us navigate to elements in our HTML by `id`.

# HTML IDs

```
<h2 id="visited">Visited</h2>
<p id="intro">Here is some text about the purpose of my made-up blog.</p>
<div id="footer">Copyright 2026</div>
```

- `id` is an attribute, like `class`, and any element can have one.
- Its value is a name for that element, and the name must be **unique** within the document: no two elements share an `id`.

## Problem: Two Elements, One ID

```
<p id="visited">I have visited a lot of coffee shops.</p>
<div id="visited">Visited</div>
```

Is this valid HTML?

## Solution: Two Elements, One ID

No. An `id` must be **unique**: one element per name on a page. Two elements named `visited` would leave `href="#visited"` with nowhere definite to go.

The browser will not stop you (it quietly picks the first one), but the HTML is invalid. This is the difference from `class`: a class can go on as many elements as you like; an ID goes on exactly one.

# Selecting by ID

```
#[ID_NAME] {
  ...
}
```

```
#visited {
  color: green;
}
```

```
<h2 id="visited">Visited</h2>
```

`.` selects by class; `#` selects by ID.

## Problem: Green Links

```
<p>
  Read the <a href="reviews.html">reviews</a> or my
  <a href="about.html">about page</a>.
</p>
```

- Let's say that we don't like the default behavior for `<a>` and we want our links to be green. How can we do that?

## Solution: Green Links

```
a {
  color: green;
}
```

Read the reviews or my about page.

# Problem: About Link Without Underline

```
a {
  color: green;
}
```

```
<p>
  Read the <a href="reviews.html">reviews</a> or my
  <a href="about.html">about page</a>.
</p>
```

- Now that every link is green, let's give the "about page" link special behavior: no underline. How can we do that with text-decoration?

# Solution: About Link Without Underline

```css
a {
  color: green;
}

#about {
  text-decoration: none;
}
```

```html
<p>
  Read the <a href="reviews.html">reviews</a> or my
  <a id="about" href="about.html">about page</a>.
</p>
```

Read the reviews or my about page.

# Exercise: Use a Class Instead

```
a {
  color: green;
}
```

```
<p>
  Read the <a href="reviews.html">reviews</a> or my
  <a href="about.html">about page</a>.
</p>
```

- Remove the underline from the "about page" link using a class instead of an id.

# Exercise: Use a Class Instead (Continued)

```
.about {
  text-decoration: none;
}
```

```
<p>
  Read the <a href="reviews.html">reviews</a> or my
  <a class="about" href="about.html">about page</a>.
</p>
```

Same result as `#about`. Only the selector and the attribute changed.

# Exercise: No Class, No ID

```
a {
  color: green;
}
```

```
<p>
  Read the <a href="reviews.html">reviews</a> or my
  <a href="about.html">about page</a>.
</p>
```

- Remove the underline from the "about page" link without adding a class or an id. (Hint: a space in a selector means *inside*.)

# Exercise: No Class, No ID (Continued)

```
span a {
  text-decoration: none;
}
```

```
<p>
  Read the <a href="reviews.html">reviews</a> or my
  <span><a href="about.html">about page.</a></span>
</p>
```

# Which Is Best? Which Is Worst? Why?

```
/* id */
#about {
  text-decoration: none;
}

/* class */
.about {
  text-decoration: none;
}

/* trick */
span a {
  text-decoration: none;
}
```

## Which Is Best? Which Is Worst? Why? My Thoughts

```
/* id */
#about {
  text-decoration: none;
}
/* class */
.about {
  text-decoration: none;
}
/* trick */
span a {
  text-decoration: none;
}
```

The worst is the "trick." It wouldn't make sense to others reading the HTML. Given the name "about", I'd say `id` makes the most sense. But a renamed class like "link-no-underline" would be reusable.

## Which One?

| Selector | Use when |
| --- | --- |
| Tag `p` | Every element of that type should look the same |
| Class `.favorite` | A group of elements share a look |
| ID `#header` | Exactly one element needs its own look |

# Problem: Class and ID (Line 1)

```
.section {
  color: gray;
  background-color: yellow;
}

#visited {
  color: green;
}
```

```
<a href="#visited">Jump to visited</a>
```

What does this link look like? What happens when it is clicked?

## Solution: Class and ID (Line 1)

```
.section {
  color: gray;
  background-color: yellow;
}

#visited {
  color: green;
}
```

```
<a href="#visited">Jump to visited</a>
```

Neither rule matches: the link has no class and no id, so it keeps the default look. Clicking it scrolls to the element with `id="visited"`. If no element has that id, the address bar still changes to `#visited`, but the page does not move and no error appears.

# Problem: Class and ID (Line 2)

```
.section {
  color: gray;
  background-color: yellow;
}

#visited {
  color: green;
}
```

```
<h2 class="section" id="wishlist">Wishlist</h2>
```

What does this heading look like?

## Solution: Class and ID (Line 2)

```
.section {
  color: gray;
  background-color: yellow;
}

#visited {
  color: green;
}
```

```
<h2 class="section" id="wishlist">Wishlist</h2>
```

## Wishlist

Matches `.section`: gray text on yellow. `#visited` does not match, because this id is `wishlist`.

# Problem: Class and ID (Line 3)

```
.section {
  color: gray;
  background-color: yellow;
}

#visited {
  color: green;
}
```

```
<h2 class="section" id="visited">Visited</h2>
```

What does this heading look like? It shares `class="section"` with line 2: is that valid?

## Solution: Class and ID (Line 3)

```
.section {
  color: gray;
  background-color: yellow;
}

#visited {
  color: green;
}
```

```
<h2 class="section" id="visited">Visited</h2>
```

**Visited**

Matches both. The ID wins on `color`, so the text is green. The yellow is from `.section`. Id `visited` is unique. (This is where the link would navigate to.)

## When Rules Conflict

More than one rule can match the same element. When they disagree, CSS picks one:

- The most specific selector wins: **ID > class > tag**. If two rules are equally specific, the one further down in the CSS wins.
- Within a tier (e.g. `ID`), more pieces beat fewer: `span a` (two tags) beats `a` (one tag). A higher tier still wins first: `.about` (one class) beats `span a` (two tags)

## Problem: Two Rules, One Selector

The most specific selector wins: **ID > class > tag**. If two rules are equally specific, the one further down in the CSS wins.

```
p {
  font-size: 16px;
}

p {
  font-size: 48px;
}
```

```
<p>Will I have a font size of 16 pixels or 48 pixels?</p>
```

## Solution: Two Rules, One Selector

```
p {
  font-size: 16px;
}

p {
  font-size: 48px;
}
```

# Will I have a font size of 16 pixels or 48 pixels?

`48px`. Both selectors are the same, so neither is more specific. The tie goes to the rule further down in the CSS.

# Problem: When Rules Conflict

```
p {
  color: black;
}
.note {
  color: blue;
}
#warning {
  color: red;
}
```

```
<p class="note" id="warning">What color am I?</p>
```

All three rules match this paragraph. What color will it be?

(Hint: **ID > class > tag**)

## Solution: When Rules Conflict

```
p {
  color: black;
}
.note {
  color: blue;
}
#warning {
  color: red;
}
```

What color am I?

Red. The most specific selector wins: **ID > class > tag**. If two rules are equally specific, the one further down in the CSS wins.

## Styling Text

```
p {
  text-align: center; /* left, center, right                */
  text-transform: uppercase; /* uppercase, lowercase, capitalize */
  letter-spacing: 2px; /* space between letters              */
  line-height: 2; /* space between lines (x font size) */
}
```

HERE IS SOME TEXT ABOUT THE PURPOSE OF MY MADE-UP BLOG. IT RUNS ONTO A SECOND LINE SO THE LINE HEIGHT SHOWS.

## Problem: Centered and Uppercase

```
<h2>Visited</h2>
```

Center the heading and make it all caps.

| Property | Values |
| --- | --- |
| `text-align` | `left`, `center`, `right` |
| `text-transform` | `uppercase`, `lowercase`, `capitalize` |

## Solution: Centered and Uppercase

```
h2 {
  text-align: center;
  text-transform: uppercase;
}
```

**VISITED**

# Problem: Letter and Line Spacing

```
<h2>Visited</h2>
<p>
  Here is some text about the purpose of my made-up blog. I drink a lot of
  coffee, I have opinions about where to drink it, and I wanted somewhere to
  write those opinions down. That is the whole plan.
</p>
```

Put `2px` of space between the heading's letters, and make the paragraph's lines twice as tall as normal.

| Property | Values |
| --- | --- |
| `letter-spacing` | a length, e.g. `2px` |
| `line-height` | a number, e.g. `1.6` (a multiple of the font size) |

## Solution: Letter and Line Spacing

```
h2 {
  letter-spacing: 2px;
}

p {
  line-height: 2;
}
```

## Visited

Here is some text about the purpose of my made-up blog. I drink a lot of coffee, I have opinions about where to drink it, and I wanted somewhere to write those opinions down. That is the whole plan.

## Pseudo-Classes

So far, selectors pick elements by what they *are*: a tag, a class, an id. A **pseudo-class** picks elements by what *state* they are in:

- the mouse is over it (`:hover`)
- the user has already visited it (`:visited`)
- it is being clicked right now (`:active`)

There are many more.

# Pseudo-Class Format

```
[SELECTOR]:[STATE] {
  [property]: [VALUE];
}
```

```
a:hover {
  color: orange;
}
```

- A colon, no space: `a:hover` is a link *and* the mouse is over it, the same way `span.bold` is a span *and* has class `bold`.
- For "which rule wins", a pseudo-class counts like a class. `a:hover` is more specific than `a`.

## Problem: Hover

```
a {
  color: green;
  text-decoration: none;
}
```

```
<a href="index.html">Home</a>
<p>
  Read the <a href="reviews.html">reviews</a> or my
  <a href="about.html">about page</a>.
</p>
```

While the mouse is over a link *inside a paragraph*, make its text `48px`. The Home link is not in a paragraph and should not change on hover.

## Solution: Hover

```
p a:hover {
  font-size: 48px;
}
```

Home

Read the reviews or my about page.

Space means inside, colon means state: a link inside a `<p>` that the mouse is over.

Home is not inside a `<p>`, so it never matches.

# Problem: Visited

```
a {
  color: green;
  text-decoration: none;
}
```

```
<p>Read my <a href="about.html">about page</a>.</p>
<ul class="reviews">
  <li><a href="recreo.html">Recreo</a></li>
  <li><a href="nero.html">Nero</a></li>
</ul>
```

Browsers turn a link purple once you have visited it. Our `a` rule makes every link green, visited or not. In the reviews list only, make links the user has already visited gray, so they can see which reviews they have read.

## Solution: Visited

```
.reviews a:visited {
  color: gray;
}
```

Read my about page.

- Recreo
- Nero

Pretend the user has already read the about page and the Recreo review. Both links are visited, but only Recreo is inside `.reviews`, so only Recreo turns gray.

NOTE: `:visited` can only change colors. Browsers block anything else for privacy.

## Problem: Active

```
a {
  color: green;
  text-decoration: none;
}
```

```
<p>
  Read the <a href="reviews.html">reviews</a> or my
  <a class="about" href="about.html">about page</a>.
</p>
```

While the mouse button is held down on the about link, and only that one, make it red and uppercase.

## Solution: Active

```
.about:active {
  color: red;
  text-transform: uppercase;
}
```

```
<p>
  Read the <a href="reviews.html">reviews</a> or my
  <a class="about" href="about.html">about page</a>.
</p>
```

Read the reviews or my about page.

No space: class `about` *and* being clicked. Press and hold on it above; `:active` matches from mouse down to mouse up.

# Problem: Order Matters

```
a {
  color: green;
  text-decoration: none;
}
a:hover {
  color: orange;
}
a:visited {
  color: gray;
}
```

```
<a href="https://www.google.com">Example</a>
```

- What color is the link when the mouse is not over it?
- What color is the link when the mouse is over it?

## Solution: Order Matters

```
a {
  color: green;
  text-decoration: none;
}
a:hover {
  color: orange;
}
a:visited {
  color: gray;
}
```

- Not over it: green if you have never been to google.com, gray if you have. (You have.)
- Over it: orange if never visited. Gray if visited: `a:hover` and `a:visited` both match and are equally specific, so the one further down wins.

## Pseudo-Elements

A pseudo-class (`:hover`) selects a whole element by its state. A **pseudo-element** selects *part* of an element: its first letter, its first line, the text the user has highlighted.

```
[SELECTOR]::[PART] {
  [property]: [VALUE];
}
```

- Two colons: `p::first-letter` is the first letter of every `<p>`.
- Others you will see: `::first-line`, `::selection`, `::before`, `::after`. MDN lists them all.

## Problem: Big First Letter

```
<p>
  Here is some text about the purpose of my made-up blog. I drink a lot of
  coffee, I have opinions about where to drink it, and I wanted somewhere to
  write those opinions down. That is the whole plan.
</p>
```

Make the first letter of every paragraph `48px`, like the oversized letter that starts a book chapter. (Hint: `::first-letter` `font-size`)

## Solution: Big First Letter

```
p::first-letter {
  font-size: 48px;
}
```

Here is some text about the purpose of my made-up blog. I drink a lot of coffee, I have opinions about where to drink it, and I wanted somewhere to write those opinions down. That is the whole plan.

## Problem: Uppercase First Line

```
<p>
  Here is some text about the purpose of my made-up blog. I drink a lot of
  coffee, I have opinions about where to drink it, and I wanted somewhere to
  write those opinions down. That is the whole plan.
</p>
```

Make the first line of every paragraph uppercase. (Hint: `::first-line` `text-transform`)

## Solution: Uppercase First Line

```
p::first-line {
  text-transform: uppercase;
}
```

HERE IS SOME TEXT ABOUT THE PURPOSE OF MY MADE-UP BLOG. I DRINK A lot of coffee, I have opinions about where to drink it, and I wanted somewhere to write those opinions down. That is the whole plan.

Which words are on the first line depends on the width of the page. Make the window narrower and the uppercase part follows.

# Review: Attributes

```
<a href="about.html" target="_blank">about page</a>
<img src="cup.png" alt="A cup of coffee" />
<p class="compact" id="intro">Here is some text.</p>
```

- An **attribute** is a `name="value"` pair inside an opening tag.
- `href`, `target`, `src`, `alt`, `class`, `id`, `style` are all attributes.

## Attribute Selectors

An **attribute selector** works for any attribute: `alt`, `href`, `target`, ...

```
[SELECTOR][[ATTRIBUTE]] {
  [property]: [VALUE];
}
```

## Attribute Selectors (Continued)

```
a[href] {
  text-transform: capitalize;
}
```

```
<a href="reviews.html">reviews</a> <a>coming soon</a>
```

Reviews coming soon

`[href]` asks one question: **does this element have an `href` attribute?** The value does not matter. The first link does, so it matches; the second has no `href`, so it is left alone.

## Problem: Images Without Alt Text

```
<img src="cup.png" alt="A cup of coffee" /> <img src="beans.png" />
```

Make any image without `alt` text have `20%` opacity.

# Solution: Images Without Alt Text

```
img {
  opacity: 0.2;
}

img[alt] {
  opacity: 1;
}
```



## Solution: Images Without Alt Text (Continued)

```
img:not([alt]) {
  opacity: 0.2;
}
```



Same result in one rule. `:not()` is a pseudo-class that flips whatever is inside the parentheses: `img:not([alt])` is an image that does *not* have an `alt` attribute.

## Attribute Selectors: A Specific Value

```
[SELECTOR][[ATTRIBUTE="VALUE"]] {
  [property]: [VALUE];
}
```

```
a[href="index.html"] {
  font-weight: bold;
}
```

Add `="VALUE"` inside the brackets to require an exact value: a link *and* its `href` is exactly `index.html`. Quotes around the value.

## Problem: Links That Open a New Tab

```
<p>
  Read the <a href="reviews.html">reviews</a> or
  <a href="https://www.google.com" target="_blank">search</a>.
</p>
```

`target="_blank"` in `<a>` will cause the link to open in a new page. Say that we want to spread out the letters (`5px`) of every link that opens in a new tab. What is the CSS to do this?

## Solution: Links That Open a New Tab

```
a[target="_blank"] {
  letter-spacing: 5px;
}
```

```
<p>
  Read the <a href="reviews.html">reviews</a> or
  <a href="https://www.google.com" target="_blank">search</a>.
</p>
```

Read the reviews or search.

# Attribute Selectors: The Full Set

| Selector | Matches when the attribute... | Example |
|---|---|---|
| `[attr]` | exists | `img[alt]` |
| `[attr="v"]` | is exactly `v` | `a[target="_blank"]` |
| `[attr^="v"]` | starts with `v` | `a[href^="https"]` |
| `[attr$="v"]` | ends with `v` | `a[href$=".pdf"]` |
| `[attr*="v"]` | contains `v` anywhere | `a[href*="google"]` |
| `[attr~="v"]` | is a space-separated list containing `v` | `p[class~="compact"]` |
| `[attr\|="v"]` | is `v` or starts with `v-` | `p[lang\|="en"]` |

CSS gives you many more ways to select attributes: MDN.

# What If the CSS Is Hundreds of Lines?

```
<head>
  <style>
    body { ... }
    h1 { ... }
    h2 { ... }
    p { ... }
    ul { ... }
    li { ... }
    a { ... }
    /* 300 more lines */
  </style>
</head>
```

The `<head>` is now mostly CSS, and a second page needs all of it copied again.

Can I separate my HTML from my CSS?

# External CSS

`style.css`

```
p {
  color: red;
}
```

`index.html`

```
<head>
  <link rel="stylesheet" href="style.css" />
</head>
```

Yes. Put the rules in their own file and `<link>` it from the `<head>`. This is **external** CSS. One file styles every page that links to it.

# Three Places for CSS

| Kind | Where it goes | Use when |
|---|---|---|
| Inline | `style` attribute on one element | A one-off tweak to a single element |
| Internal | `<style>` in the `<head>` (also called embedded) | Rules for this one page |
| External | `<link rel="stylesheet" href="style.css" />` | Rules shared by every page on the site |

Inline beats the other two (unless `!important`). Between internal and external, the usual rules apply: more specific wins, and on a tie, whichever comes later in the `<head>`.

# Summary: Writing Rules

```
selector {
  property: value;
}
```

- Select by tag ( `p` ), class ( `.compact` ), id ( `#about` ), or attribute ( `a[target="_blank"]` )
- No space means *and* ( `span.bold` ), a space means *inside* ( `p a` ), a comma means *each of these* ( `span, a` )
- `:hover` , `:visited` , `:active` select by state; `::first-letter` , `::first-line` select part of an element
- An id is unique on the page; a class can go on as many elements as you like
- There are hundreds of properties. Learn the pattern, then look them up on MDN

## Summary: Which Rule Wins

- Children inherit text and color properties from their parent; a rule on the child itself wins
- Inline beats internal and external, unless a rule says `!important`
- The most specific selector wins: **ID > class > tag** (a pseudo-class counts like a class)
- Within a tier, more pieces beat fewer: `span a` beats `a`
- On a tie, the rule further down in the CSS wins